Physics Graduate Student Database — summary for IT

Purpose. A web application for the Physics Department to keep track of PhD students: records, advisors and committees, milestones, core-course credit, funding by term, and the candidacy-form approval process. Used by department staff, faculty (read-only view of their own students), financial services reps (FSRs), an approver and the department chair.

Owner / contact: [name, email] **Backup contact:** [name, email] **Source code:** github.com/dwschmitz/Physics-Department-DB

Architecture

- **One Python 3 program** (Flask; tested on Python 3.13) that serves HTML pages. No JavaScript framework, no external services other than optional outbound SMTP.
- **One SQLite database file** (`physics_grads.db`) holds everything: records, login accounts, generated form PDFs and the chair's signature image. Only one process writes at a time; expected load is a handful of simultaneous users.
- **Libraries:** Flask (with Werkzeug and Jinja2), openpyxl (Excel import), pypdf and Pillow (PDF forms and signature image). About 7,000 lines of code and about 175 automated tests.
- **Configuration:** environment variables or a `.env` file (mail server, session secret, public URL). Passwords and secrets are never stored in the repository or the database.

Data stored and who sees it

Student names, University ID (UCID), CNET, email, citizenship, status, matriculation / transfer / graduation / withdrawal terms, research area, internal notes, advisors and committees, milestones, core-course credit, funding (type, source, account number, amount); faculty and FSR names and emails; account usernames and password hashes. These are **student education records (FERPA)**. Nothing is sent outside the university except notification emails, which contain only a count and a link, never student names or IDs.

Role	Can do
Viewer	Read everything except account management
Editor	Viewer + add, change, delete records, import students from a file
Administrator	Editor + manage accounts
Approver	Editor + first-round approval of candidacy forms
Chair	Administrator + approval and the final signature on candidacy forms
Faculty	Only their own advisees and committee students; never UCIDs or internal notes (default-deny: every other page is refused)
FSR	Only the advisees of the faculty they support; read-only

Security controls in place

Passwords hashed with scrypt, minimum 10 characters, temporary passwords must be changed at first login; changing or resetting a password signs that person out everywhere. CSRF token on every change. Session cookies are HttpOnly and SameSite=Lax; sessions last 8 hours. Database queries use bound parameters; page output is escaped. Uploads are size-limited (5 MB) and type-checked; signature images are re-encoded; filled-in PDFs are flattened (not editable). Form PDFs and the signature image are only available to permitted roles. Role rules for faculty, FSR, approver and chair are enforced on the server and covered by tests. **Sign-in is throttled** (5 failures per username or 30 per address within 15 minutes block further attempts for up to 15 minutes). When served over HTTPS (`PHYSDB_HTTPS=1`) cookies are marked **Secure** and HSTS is sent. An **audit log** (database triggers) records who changed which record, when, and the before/after values, plus sign-ins and lockouts; it is viewable by administrators and the chair, and never contains passwords. Every page carries a **strict Content-Security-Policy** (no inline scripts or styles), X-Frame-Options DENY, nosniff, a same-origin referrer policy and no-store caching.

Known gaps (planned before production use)

- **Sign-in is local passwords, not university single sign-on, and has no two-factor step.** Usernames are already CNETs, so SSO is a natural next step (we need to know which service to integrate).
- Runs on the development web server by default; a production WSGI server (e.g. gunicorn behind nginx) is needed. No built-in backups or high availability. Accessibility has not been formally reviewed.

What we need from IT

1. A small Linux VM or container (1 vCPU, 1 GB RAM, a few GB of disk; the data is megabytes), a current Python 3 (developed and tested on 3.13).
2. A hostname and TLS certificate; a reverse proxy; access limited to campus network or VPN at first, if acceptable.
3. Encrypted storage and regular backups of `physics_grads.db`, `.env` and `.secret_key`, with a tested restore.
4. SSO (SAML/Shibboleth or OIDC) registration, and an SMTP relay / sending address for notices.
5. Their security-review and data-classification process, and who patches the OS and the Python libraries.